

MANUAL DO AMBIENTE

Power BI -> Web

Guia simplificado para migrar um dashboard Power BI até uma aplicação web em Next.js, com dados do Snowflake (camada ouro).

diagnostico -> DAX review -> mapear -> gerar -> validar -> auth -> deploy

Boilerplate powerbi-to-web | Versão 1.1 | Uso interno

1. O que é este ambiente

Um **kit de migração**: além do app Next.js, traz scripts de automação, prompts/skills de IA, componentes prontos, camada segura para o Snowflake e validação de paridade. O objetivo é recriar dashboards do Power BI na web de forma **padronizada, rastreável e segura**.

O caminho, do início ao fim

Etapa	O que acontece
1. Exportar PBIP	O .pbix vira projeto legível (TMDL + PBIR).
2. Checar TMDL/PBIR	Confirmar que os arquivos estão completos.
3. Diagnóstico (IA)	Mapear páginas, visuais, medidas, filtros e riscos.
4. Revisar DAX complexo	DAX Review Layer: classificar e validar medidas que dependem de contexto de filtro antes de virar SQL.
5. Mapear Snowflake	Ligar medidas a tabelas/views da camada ouro.
6. Gerar a página	Recriar a tela em Next.js com componentes prontos.
7. Validar paridade	Comparar números web x Power BI.
8. Proteger com auth	Login Google restrito ao domínio corporativo (Auth.js).
9. Deploy + DataHub	Publicar na Vercel e catalogar.

Conceito	O que significa
Fonte de verdade	TMDL/PBIR explicam o dashboard original. A IA usa isso para entender - nunca para adivinhar.
Dados reais	Snowflake (camada ouro), conectado SÓ no servidor. Mocks são apenas fallback de desenvolvimento.

Regra de ouro: a IA acelera o trabalho, mas **não inventa** regra de negócio nem nome de tabela/coluna/medida. Sem evidência, marca-se CHANGE_ME e registra-se a pendência.

2. Antes de começar

Você precisa de

Ferramenta	Para quê
Node.js >= 20 e npm	Rodar a aplicação e os scripts
Git	Versionar a migração
Power BI Desktop	Abrir o .pbix e exportar o PBIP
IDE com IA (VS Code / Cursor / Claude Code)	Rodar as skills e prompts
Acesso ao Snowflake (role/warehouse/schema + leitura na camada ouro)	Dados reais
Vercel CLI (npm i -g vercel)	Deploy (opcional até o fim)
Python >= 3.10	Validação de paridade (opcional)

3. Quickstart

```
# 1) Clonar e instalar
git clone <repo-template>
cd powerbi-to-web-boilerplate
npm install

# 2) Conferir o ambiente (diz, em portugues, o que falta)
npm run doctor

# 3) Configurar credenciais do Snowflake (nao versionar)
cp .env.example .env.local # preencha as variaveis SNOWFLAKE_*

# 4) Rodar localmente
npm run dev # http://localhost:3000
```

Sem o Snowflake configurado, o dashboard de exemplo usa dados *mock* - útil para ver a UI antes de conectar.

Pouca experiência com dev? Use `npm run doctor` (mostra o que falta) e `npm run migrate` (wizard que cria a estrutura e indica o proximo passo).

4. Passo a passo da migração

Substitua <slug> pelo nome do dashboard em minúsculas com hífens (ex.: vendas-por-hora).

1 Exportar o dashboard para PBIP

No Power BI Desktop, habilite PBIP, TMDL e PBIR (Preview features) e salve como .pbip. Devem surgir as pastas .Report e .SemanticModel.

2 Iniciar a estrutura do dashboard

```
npm run dashboard:init -- --slug <slug>
```

Cria pastas e arquivos base a partir do modelo _template. Depois, copie o PBIP para powerbi-input/<slug>/.

3 Checar TMDL/PBIR

```
npm run pbip:check -- --slug <slug>
npm run pbip:inventory -- --slug <slug>
```

O check valida se o pacote está completo. O inventory gera um resumo em docs/dashboards/<slug>/inventario-pbip.md.

Apareceu model.bim ou report.json? É formato legado. Reexporte com TMDL e PBIR habilitados.

4 Diagnóstico com IA

Na IDE, acione a skill pbip-diagnostico (ou cole prompts/01). A IA lê o TMDL/PBIR e preenche o diagnóstico: páginas, visuais, medidas, filtros, riscos e complexidade.

5 Revisar medidas DAX complexas (DAX Review Layer)

```
npm run dax:extract -- --slug <slug>          # medidas do TMDL -> catalogo
npm run dax:classify -- --slug <slug>          # simple|moderate|complex|critical
npm run dax:review-cards -- --slug <slug>      # cards + traducoes + casos
```

Acione a skill dax-review (ou prompts/03). Medidas que dependem do contexto de filtro (CALCULATE, SELECTEDVALUE, REMOVEFILTERS, ALL/ALLSELECTED, iteradores, inteligência de tempo) são revisadas **antes** de virar SQL.

Prefira **bloquear** uma tradução insegura a gerar SQL incorreto: medida crítica vira blocked_for_auto_translation até ter caso de validação que comprove a paridade.

6 Configurar o Snowflake

```
npm run snowflake:check # confere variaveis (sem mostrar valores)
npm run snowflake:test  # SELECT CURRENT_VERSION()
```

As variáveis ficam em .env.local (local) e nas Environment Variables da Vercel (deploy).

4. Passo a passo da migração (continuação)

7

Mapear a camada ouro do Snowflake

Acione snowflake-mapeamento (ou prompts/04). A IA liga cada medida/visual a uma tabela/view da camada ouro e cria o source-map, as queries SQL, os adapters e o rastreamento de métricas.

Sem evidência do nome real, usa-se CHANGE_ME_* e marca-se a pendência.

8

Gerar a página web

Acione gerar-dashboard-web (ou prompts/05 e 06). A IA recria a tela com os componentes prontos (header, filtros, KPIs, gráfico Recharts, tabela) e os estados de carregamento, erro e vazio.

```
npm run dev # http://localhost:3000/dashboards/<slug>
```

9

Validar a paridade contra o Power BI

```
npm run validation:init -- --slug <slug>
python validation/compare-results.py \
  --expected validation/dashboards/<slug>/powerbi/expected-results/kpis.csv \
  --actual validation/dashboards/<slug>/web/api-results/kpis.csv \
  --key METRIC_ID --value VALUE --tolerance 0.01
```

Compara os números (diferença absoluta, %, status). A validação final é de **negócio**: alguém da área assina.

10

Proteger com autenticação

```
npx auth secret # gera AUTH_SECRET em .env.local
# AUTH_GOOGLE_ID / AUTH_GOOGLE_SECRET / AUTH_ALLOWED_DOMAINS
```

Acione auth-nextauth (ou prompts/08–10). Todo dashboard nasce protegido por **Auth.js + Google**, com o middleware.ts restringindo o acesso ao domínio corporativo. O painel embutido no DataHub (iframe) continua autenticado.

Nunca exponha as variáveis AUTH_* no client nem desabilite o middleware.ts sem autorização.

11

Deploy na Vercel

```
npm run lint && npm run typecheck && npm run test && npm run build
vercel # preview
vercel --prod # producao
```

Configure as variáveis SNOWFLAKE_* e AUTH_* no painel da Vercel.

12

Publicar no DataHub

Abra um PR usando o template do repositório e publique/embuta a URL do dashboard no DataHub conforme o processo da sua área.

5. Skills de IA do ambiente

As skills ficam em `.claude/skills/` e **disparam sozinhas** quando o pedido casa com a descrição - ou você as chama por `/<nome>`.

Skill	Quando usar
migrar-dashboard	Porta de entrada - conduz o fluxo inteiro
pbip-diagnostico	Checar/inventariar o PBIP e gerar o diagnóstico
dax-review	Revisar/classificar/validar medidas DAX complexas antes do SQL
snowflake-mapeamento	Mapear medidas -> camada ouro, queries, adapters
gerar-dashboard-web	Recriar a página em Next.js
validar-paridade	Comparar números web x Power BI
auth-nextauth	Proteger com Auth.js + Google (domínio, iframe DataHub)
deploy-vercel	Deploy + variáveis de ambiente
recharts	Padrões de gráfico do projeto
snowflake-semanticview	Criar/validar semantic views via snow CLI

6. Regras de ouro

Segurança: credenciais nunca vão para o navegador. Não use `NEXT_PUBLIC_` para segredos. A conexão Snowflake acontece só no servidor.

Autenticação: não exponha as variáveis `AUTH_*` no client nem desabilite o `middleware.ts` (login Google restrito ao domínio corporativo) sem autorização. Auth só em `src/server/auth`.

Sem invenção: não crie regra de negócio nem nome de tabela/coluna sem evidência no TMDL/PBIR. Use `CHANGE_ME` e documente em `validation-notes.md`.

Rastreabilidade: toda métrica deve ser rastreável entre Power BI, Snowflake e componente web. Mocks só como fallback.

7. Checklist final

☐ PBIP copiado para powerbi-input/<slug>/	☐ Sem segredos no frontend
☐ TMDL e PBIR encontrados (pbip:check)	☐ Auth ativa (middleware + domínio + login Google)
☐ Diagnóstico criado	☐ lint, typecheck, test, build ok
☐ Medidas DAX complexas revisadas (cards)	☐ Mocks identificados (se houver)
☐ Nenhuma medida crítica bloqueada pendente	☐ Paridade validada (ou pendência documentada)
☐ Source-map sem CHANGE_ME pendente	☐ Deploy preview criado
☐ Queries apontando para a camada ouro real	☐ PR aberto + publicado no DataHub
☐ Página criada (KPIs, gráfico, tabela, estados)	

8. Problemas comuns

Sintoma	O que fazer
Dashboard mostra 'Dados: Mock'	Snowflake não configurado. Rode snowflake:check e preencha .env.local.
pbip:check acusa TMDL ausente	Reexporte o PBIP com TMDL habilitado no Power BI Desktop.
snowflake:test falha	Verifique role/warehouse/schema e a chave privada (PEM).
Página não aparece em /dashboards/_template	Pastas com '_' são privadas (modelo). Crie um slug real com dashboard:init.
Números não batem com o Power BI	Confira filtros e período iguais; ajuste a query/adapter; registre tolerância.
Medida complexa diverge (CALCULATE etc.)	Revise o card da DAX Review Layer (contexto de filtro) antes de mexer na query.
Sempre cai em /auth/signin	Faça login com a conta corporativa; confira AUTH_* e AUTH_ALLOWED_DOMAINS.

9. Referência rápida de comandos

```
npm run doctor           # checa o ambiente
npm run migrate          # wizard guiado (recomendado)
npm run dashboard:init -- --slug <slug>  # cria a estrutura
npm run pbip:check -- --slug <slug>      # valida o PBIP
npm run pbip:inventory -- --slug <slug>  # inventario
npm run validation:init -- --slug <slug>  # pastas de validacao
npm run snowflake:check | snowflake:test # conexao
npm run dev | lint | typecheck | test | build
vercel | vercel --prod      # deploy
```

Documentação completa em docs/00...14 | Prompts em prompts/00...11 | Regras para IA em AGENTS.md / CLAUDE.md